



Assignment - 1

Aim:- To Implement insertion & preorder traversal operations in a BST.

Algorithm:-

1. Search and add node in a BST steps:

1. Start from root of the BST
2. If root is Null, create new node & insert the value.
3. If new value equals the value of the root, the value already exists.
4. If new value $>$ root value
 recursively search in Right subtree.
5. else
 recursively search in Left subtree
6. Insert the new Node & Return.

2. Search maximum Value in BST steps:

1. Start from root the Node
2. If root is NULL, tree is empty. return
3. Move the right child as long as it exists.
4. The rightmost child contains maximum value
 return the rightmost child in the
 and leaf node



COEP TECHNOLOGICAL UNIVERSITY, PUNE

Wellesley Road, Shivajinagar, Pune-411 005

3. Preorder traversal with recursion

steps:

- i. If root is Null, return.
- ii. Visit & print the root Node.
- iii. Recursively traverse the right subtree.

4. Preorder traversal without recursion

steps:

- i. if root is null, return.
- ii. Initialise an empty stack & push root into it.
- iii. While stack is not empty.
 - pop the top node & print it.
 - push right child if exists.
 - push left child if exists.

5. Levelorder(root)

(i) enqueue root Node.

(ii) while (queue is not empty)

- dequeue all elements of queue & print them.
- and enqueue their non-null children
- print level number and new line.

properties:- BST is a non-linear data structure with each node having between [0, 2] nodes as children (left, right) of parent node. In BST the values are in order of left child < parent < right child.

Conclusion:-

Successfully implemented Binary Search Tree for Integer data values and printed all traversal Pre orders.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <limits.h>
4
5 // --- Structure Definitions ---
6
7 typedef struct Node {
8     int val;
9     struct Node* left;
10    struct Node* right;
11 } Node;
12
13 typedef struct Stack {
14     Node** data;
15     int top;
16     int capacity;
17 } Stack;
18
19 typedef struct Queue {
20     Node** data;
21     int front;
22     int rear;
23     int capacity;
24 } Queue;
25
26 // --- Node Helper Functions ---
27
28 Node* create_node(int val) {
29     Node* new_node = (Node*)malloc(sizeof(Node));
30     if (!new_node) return NULL;
31     new_node->val = val;
32     new_node->left = NULL;
33     new_node->right = NULL;
34     return new_node;
35 }
36
37 void free_tree(Node* root) {
38     if (root == NULL) return;
39     free_tree(root->left);
40     free_tree(root->right);
41     free(root);
42 }
43
44 // --- Tree Operations ---
45
46 void add(int data, Node** root) {
47     if (*root == NULL) {
48         *root = create_node(data);
49         return;
50     }
51
52     Node* temp = *root;
53     while (1) {
54         if (data < temp->val) {

```



```

52 Node* temp = *root;
53 while (1) {
54     if (data < temp->val) {
55         if (temp->left == NULL) {
56             temp->left = create_node(data);
57             return;
58         }
59         temp = temp->left;
60     } else if (data > temp->val) {
61         if (temp->right == NULL) {
62             temp->right = create_node(data);
63             return;
64         }
65         temp = temp->right;
66     } else {
67         return; // Duplicate value
68     }
69 }
70 }

```

```

71
72 Node* search(Node* root, int data) {
73     Node* temp = root;
74     while (temp != NULL) {
75         if (data < temp->val)
76             temp = temp->left;
77         else if (data > temp->val)
78             temp = temp->right;
79         else
80             return temp;
81     }
82     return NULL;
83 }

```

```

84
85 int maxi(Node* root) {
86     if (root == NULL)
87         return INT_MIN;
88
89     while (root->right != NULL)
90         root = root->right;
91
92     return root->val;
93 }

```

```

94
95 int mini(Node* root) {
96     if (root == NULL)
97         return INT_MAX;
98
99     while (root->left != NULL)
100         root = root->left;
101
102     return root->val;
103 }

```

```

104
105 static inline int max_int(int a, int b) {

```

```

105 static inline int max_int(int a, int b) {
106     return (a > b) ? a : b;
107 }
108
109 int height(Node* root) {
110     if (root == NULL)
111         return 0;
112
113     return 1 + max_int(height(root->left), height(root->right));
114 }
115
116 // --- Recursive Traversals ---
117
118 void inorder(Node* root) {
119     if (root == NULL)
120         return;
121
122     inorder(root->left);
123     printf("%d ", root->val);
124     inorder(root->right);
125 }
126
127 void preorderRec(Node* root) {
128     if (root == NULL)
129         return;
130
131     printf("%d ", root->val);
132     preorderRec(root->left);
133     preorderRec(root->right);
134 }
135
136 void postorder(Node* root) {
137     if (root == NULL)
138         return;
139
140     postorder(root->left);
141     postorder(root->right);
142     printf("%d ", root->val);
143 }
144
145 // --- Stack Utilities & Iterative Preorder Traversal ---
146
147 Stack* create_stack(int capacity) {
148     Stack* st = (Stack*)malloc(sizeof(Stack));
149     st->capacity = capacity;
150     st->top = -1;
151     st->data = (Node**)malloc(sizeof(Node*) * capacity);
152     return st;
153 }
154
155 void push(Stack* st, Node* node) {
156     if (st->top == st->capacity - 1) {
157         st->capacity *= 2;
158         st->data = (Node**)realloc(st->data, sizeof(Node*) * st->capacity)

```

```

157     st->capacity *= 2;
158     st->data = (Node**)realloc(st->data, sizeof(Node*) * st->capacity)
159 }
160 st->data[++st->top] = node;
161 }
162
163 Node* pop(Stack* st) {
164     if (st->top == -1) return NULL;
165     return st->data[st->top--];
166 }
167
168 int is_stack_empty(Stack* st) {
169     return st->top == -1;
170 }
171
172 void free_stack(Stack* st) {
173     free(st->data);
174     free(st);
175 }
176
177 void preorderIte(Node* root) {
178     if (root == NULL)
179         return;
180
181     Stack* st = create_stack(16);
182     push(st, root);
183
184     while (!is_stack_empty(st)) {
185         Node* curr = pop(st);
186
187         printf("%d ", curr->val);
188
189         if (curr->right)
190             push(st, curr->right);
191         if (curr->left)
192             push(st, curr->left);
193     }
194
195     free_stack(st);
196 }
197
198 // --- Queue Utilities & Levelorder / Tree Visualization ---
199
200 Queue* create_queue(int capacity) {
201     Queue* q = (Queue*)malloc(sizeof(Queue));
202     q->capacity = capacity;
203     q->front = 0;
204     q->rear = 0;
205     q->data = (Node**)malloc(sizeof(Node*) * capacity);
206     return q;
207 }
208
209 void enqueue(Queue* q, Node* node) {
210     if (q->rear == q->capacity) {

```



```

302
303 int main() {
304     Node* root = NULL;
305
306     // Building the binary search tree
307     add(10, &root);
308     add(5, &root);
309     add(15, &root);
310     add(3, &root);
311     add(7, &root);
312     add(12, &root);
313     add(18, &root);
314
315     printf("Visualized Tree Layout:\n");
316     printTree(root);
317
318     printf("Inorder Traversal: ");
319     inorder(root);
320     printf("\n");
321
322     printf("Preorder Recursive: ");
323     preorderRec(root);
324     printf("\n");
325
326     printf("Preorder Iterative: ");
327     preorderIte(root);
328     printf("\n");
329
330     printf("Postorder Traversal: ");
331     postorder(root);
332     printf("\n");
333
334     printf("Levelorder Traversal: ");
335     levelorder(root);
336     printf("\n");
337
338     printf("\nTree Statistics:\n");
339     printf("Height: %d\n", height(root));
340     printf("Min Value: %d\n", mini(root));
341     printf("Max Value: %d\n", maxi(root));
342
343     int target = 7;
344     Node* result = search(root, target);
345     if (result) {
346         printf("Search %d: Found\n", target);
347     } else {
348         printf("Search %d: Not Found\n", target);
349     }
350
351     // Clean up allocated tree nodes
352     free_tree(root);
353
354     return 0;
355 }

```

```

209 void enqueue(Queue* q, Node* node) {
210     if (q->rear == q->capacity) {
211         q->capacity *= 2;
212         q->data = (Node**)realloc(q->data, sizeof(Node*) * q->capacity);
213     }
214     q->data[q->rear++] = node;
215 }
216
217 Node* dequeue(Queue* q) {
218     if (q->front == q->rear) return NULL;
219     return q->data[q->front++];
220 }
221
222 int queue_size(Queue* q) {
223     return q->rear - q->front;
224 }
225
226 int is_queue_empty(Queue* q) {
227     return q->front == q->rear;
228 }
229
230 void free_queue(Queue* q) {
231     free(q->data);
232     free(q);
233 }
234
235 void levelorder(Node* root) {
236     if (root == NULL)
237         return;
238
239     Queue* q = create_queue(16);
240     enqueue(q, root);
241
242     while (!is_queue_empty(q)) {
243         Node* curr = dequeue(q);
244
245         printf("%d ", curr->val);
246
247         if (curr->left)
248             enqueue(q, curr->left);
249         if (curr->right)
250             enqueue(q, curr->right);
251     }
252
253     free_queue(q);
254 }
255
256 void printTree(Node* root) {
257     if (root == NULL)
258         return;
259
260     int h = height(root);
261     Queue* q = create_queue(32);
262     enqueue(q, root);

```



```

256 void printTree(Node* root) {
257     if (root == NULL)
258         return;
259
260     int h = height(root);
261     Queue* q = create_queue(32);
262     enqueue(q, root);
263
264     int level = 0;
265
266     while (!is_queue_empty(q)) {
267         int nodes = queue_size(q);
268         int firstSpace = (1 << (h - level)) - 1;
269         int betweenSpace = (1 << (h - level + 1)) - 1;
270
271         for (int i = 0; i < firstSpace; i++)
272             printf(" ");
273
274         while (nodes--) {
275             Node* curr = dequeue(q);
276
277             if (curr) {
278                 printf("%d", curr->val);
279                 enqueue(q, curr->left);
280                 enqueue(q, curr->right);
281             } else {
282                 printf(" ");
283                 enqueue(q, NULL);
284                 enqueue(q, NULL);
285             }
286
287             for (int i = 0; i < betweenSpace; i++)
288                 printf(" ");
289         }
290
291         printf("\n\n");
292
293         level++;
294         if (level == h)
295             break;
296     }
297
298     free_queue(q);
299 }
300
301 // --- Main Executable ---
302
303 int main() {
304     Node* root = NULL;
305
306     // Building the binary search tree
307     add(10, &root);
308     add(5, &root);
309     add(15, &root);

```




Part B] Binary Search Tree (strings)

Aim:-

To design, implement a program for BST storing string values via linked list. The Algorithm handles string search and insertion based on lexicographical order.

Algorithm:-

i] Search and Insertion (root, str)
- if (root == Null) root = new Node(str);
- compare str with curr node via strcmp();
- if (strcmp < 0) go left node
 strcmp > 0 go right node
 else return (ignore duplicates).

ii] Level order printing (root)

① - enqueue root Node.

- while queue ≠ empty

- count elements in queue.

- dequeue those many nodes to print and enqueue their children if exists.

- print new line with level number.

iii] Preorder (root)

- if (node == null) return

- print curr node string

- call preorder (curr → left) if exists.

- call preorder (curr → right) if exists.



(iv) preorder (root) (iterative)

- if root == null return
- push root in stack.
- while (stack $\neq$ empty)
 - pop stack top and print node string
 - push (curr $\rightarrow$ right) if exists
 - push (curr $\rightarrow$ left) if exists.


```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 // --- Structure Definitions ---
6
7 typedef struct Node {
8     char* val;
9     struct Node* left;
10    struct Node* right;
11 } Node;
12
13 typedef struct Stack {
14     Node** data;
15     int top;
16     int capacity;
17 } Stack;
18
19 typedef struct Queue {
20     Node** data;
21     int front;
22     int rear;
23     int capacity;
24 } Queue;
25
26 // --- Node Helper Functions ---
27
28 Node* create_node(const char* val) {
29     Node* new_node = (Node*)malloc(sizeof(Node));
30     if (!new_node) return NULL;
31
32     // Allocate memory for the string length + 1 for null terminator
33     new_node->val = (char*)malloc(strlen(val) + 1);
34     if (!new_node->val) {
35         free(new_node);
36         return NULL;
37     }
38
39     strcpy(new_node->val, val);
40     new_node->left = NULL;
41     new_node->right = NULL;
42     return new_node;
43 }
44 void free_tree(Node* root) {
45     if (root == NULL) return;
46     free_tree(root->left);
47     free_tree(root->right);
48     free(root->val); // Free dynamically allocated string memory
49     free(root);
50 }
51
52 // --- Tree Operations ---
53
54 void add(const char* data, Node** root) {

```



```

53
54 void add(const char* data, Node** root) {
55     if (*root == NULL) {
56         *root = create_node(data);
57         return;
58     }
59
60     Node* temp = *root;
61     while (1) {
62         int cmp = strcmp(data, temp->val);
63         if (cmp < 0) {
64             if (temp->left == NULL) {
65                 temp->left = create_node(data);
66                 return;
67             }
68             temp = temp->left;
69         } else if (cmp > 0) {
70             if (temp->right == NULL) {
71                 temp->right = create_node(data);
72                 return;
73             }
74             temp = temp->right;
75         } else {
76             return; // Duplicate string value
77         }
78     }
79 }
80
81 Node* search(Node* root, const char* data) {
82     Node* temp = root;
83     while (temp != NULL) {
84         int cmp = strcmp(data, temp->val);
85         if (cmp < 0)
86             temp = temp->left;
87         else if (cmp > 0)
88             temp = temp->right;
89         else
90             return temp;
91     }
92     return NULL;
93 }
94
95 const char* maxi(Node* root) {
96     if (root == NULL)
97         return NULL;
98
99     while (root->right != NULL)
100         root = root->right;
101
102     return root->val;
103 }
104
105 const char* mini(Node* root) {
106     if (root == NULL)

```

```

105 const char* mini(Node* root) {
106     if (root == NULL)
107         return NULL;
108
109     while (root->left != NULL)
110         root = root->left;
111
112     return root->val;
113 }
114
115 static inline int max_int(int a, int b) {
116     return (a > b) ? a : b;
117 }
118
119 int height(Node* root) {
120     if (root == NULL)
121         return 0;
122
123     return 1 + max_int(height(root->left), height(root->right));
124 }
125
126 // --- Recursive Traversals ---
127
128 void inorder(Node* root) {
129     if (root == NULL)
130         return;
131
132     inorder(root->left);
133     printf("%s ", root->val);
134     inorder(root->right);
135 }
136
137 void preorderRec(Node* root) {
138     if (root == NULL)
139         return;
140
141     printf("%s ", root->val);
142     preorderRec(root->left);
143     preorderRec(root->right);
144 }
145
146 void postorder(Node* root) {
147     if (root == NULL)
148         return;
149
150     postorder(root->left);
151     postorder(root->right);
152     printf("%s ", root->val);
153 }
154
155 // --- Stack Utilities & Iterative Preorder Traversal ---
156
157 Stack* create_stack(int capacity) {
158     Stack* st = (Stack*)malloc(sizeof(Stack));

```



```

150
157 Stack* create_stack(int capacity) {
158     Stack* st = (Stack*)malloc(sizeof(Stack));
159     st->capacity = capacity;
160     st->top = -1;
161     st->data = (Node**)malloc(sizeof(Node*) * capacity);
162     return st;
163 }
164
165 void push(Stack* st, Node* node) {
166     if (st->top == st->capacity - 1) {
167         st->capacity *= 2;
168         st->data = (Node**)realloc(st->data, sizeof(Node*) * st->capacity)
169     }
170     st->data[++st->top] = node;
171 }
172
173 Node* pop(Stack* st) {
174     if (st->top == -1) return NULL;
175     return st->data[st->top--];
176 }
177
178 int is_stack_empty(Stack* st) {
179     return st->top == -1;
180 }
181
182 void free_stack(Stack* st) {
183     free(st->data);
184     free(st);
185 }
186
187 void preorderIte(Node* root) {
188     if (root == NULL)
189         return;
190
191     Stack* st = create_stack(16);
192     push(st, root);
193
194     while (!is_stack_empty(st)) {
195         Node* curr = pop(st);
196
197         printf("%s ", curr->val);
198
199         if (curr->right)
200             push(st, curr->right);
201         if (curr->left)
202             push(st, curr->left);
203     }
204
205     free_stack(st);
206 }
207
208 // --- Queue Utilities & Levelorder / Tree Visualization ---
209
210 Queue* create_queue(int capacity) {

```

```

210 Queue* create_queue(int capacity) {
211     Queue* q = (Queue*)malloc(sizeof(Queue));
212     q->capacity = capacity;
213     q->front = 0;
214     q->rear = 0;
215     q->data = (Node**)malloc(sizeof(Node*) * capacity);
216     return q;
217 }
218
219 void enqueue(Queue* q, Node* node) {
220     if (q->rear == q->capacity) {
221         q->capacity *= 2;
222         q->data = (Node**)realloc(q->data, sizeof(Node*) * q->capacity);
223     }
224     q->data[q->rear++] = node;
225 }
226
227 Node* dequeue(Queue* q) {
228     if (q->front == q->rear) return NULL;
229     return q->data[q->front++];
230 }
231
232 int queue_size(Queue* q) {
233     return q->rear - q->front;
234 }
235
236 int is_queue_empty(Queue* q) {
237     return q->front == q->rear;
238 }
239
240 void free_queue(Queue* q) {
241     free(q->data);
242     free(q);
243 }
244
245 void levelorder(Node* root) {
246     if (root == NULL)
247         return;
248
249     Queue* q = create_queue(16);
250     enqueue(q, root);
251
252     while (!is_queue_empty(q)) {
253         Node* curr = dequeue(q);
254
255         printf("%s ", curr->val);
256
257         if (curr->left)
258             enqueue(q, curr->left);
259         if (curr->right)
260             enqueue(q, curr->right);
261     }
262
263     free_queue(q);

```



```

264 }
265
266 void printTree(Node* root) {
267     if (root == NULL)
268         return;
269
270     int h = height(root);
271     Queue* q = create_queue(32);
272     enqueue(q, root);
273
274     int level = 0;
275
276     while (!is_queue_empty(q)) {
277         int nodes = queue_size(q);
278         int firstSpace = (1 << (h - level)) - 1;
279         int betweenSpace = (1 << (h - level + 1)) - 1;
280
281         for (int i = 0; i < firstSpace; i++)
282             printf(" ");
283
284         while (nodes-- > 0) {
285             Node* curr = dequeue(q);
286
287             if (curr) {
288                 printf("%s", curr->val);
289                 enqueue(q, curr->left);
290                 enqueue(q, curr->right);
291             } else {
292                 printf(" ");
293                 enqueue(q, NULL);
294                 enqueue(q, NULL);
295             }
296
297             for (int i = 0; i < betweenSpace; i++)
298                 printf(" ");
299         }
300
301         printf("\n\n");
302
303         level++;
304         if (level == h)
305             break;
306     }
307
308     free_queue(q);
309 }
310
311 // --- Main Executable ---
312
313 int main() {
314     Node* root = NULL;
315
316     // Building the binary search tree with strings (Lexicographical order
317     add("Mango", &root);

```

```

313 int main() {
314     Node* root = NULL;
315
316     // Building the binary search tree with strings (Lexicographical order)
317     add("Mango", &root);
318     add("Apple", &root);
319     add("Peach", &root);
320     add("Banana", &root);
321     add("Grape", &root);
322     add("Orange", &root);
323     add("Plum", &root);
324
325     printf("Visualized Tree Layout:\n");
326     printTree(root);
327
328     printf("Inorder Traversal (Alphabetical Order): ");
329     inorder(root);
330     printf("\n");
331
332     printf("Preorder Recursive: ");
333     preorderRec(root);
334     printf("\n");
335
336     printf("Preorder Iterative: ");
337     preorderIte(root);
338     printf("\n");
339
340     printf("Postorder Traversal: ");
341     postorder(root);
342     printf("\n");
343
344     printf("Levelorder Traversal: ");
345     levelorder(root);
346     printf("\n");
347
348     printf("\nTree Statistics:\n");
349     printf("Height: %d\n", height(root));
350     printf("Min Value (Lexicographically First): %s\n", mini(root));
351     printf("Max Value (Lexicographically Last): %s\n", maxi(root));
352
353     const char* target = "Grape";
354     Node* result = search(root, target);
355     if (result) {
356         printf("Search \"%s\": Found\n", target);
357     } else {
358         printf("Search \"%s\": Not Found\n", target);
359     }
360
361     // Clean up allocated tree nodes and string buffers
362     free_tree(root);
363
364     return 0;
365 }
366

```